Multilevel Queue Feedback

Input:

```
#include <stdio.h>
#include <string.h>

#define MAX 100
#define TQ 2

struct Process {
    char pid[10];
    int at, bt, rt, type;
    int ct, tat, wt;
    int done;
};

int main() {
    struct Process p[MAX];
    int n;

    printf("Enter number of processes: ");
    scanf("%d", &n);

    for (int i = 0; i < n; i++) {
        printf("\nProcess %d\n", i + 1);

        printf("Enter PID: ");
        scanf("%s", p[i].pid);

        printf("Enter Arrival Time: ");
        scanf("%d", &p[i].at);
```

```
    printf("Enter Burst Time: ");
    scanf("%d", &p[i].bt);

    printf("Enter Type (1=System, 2=User): ");
    scanf("%d", &p[i].type);

    p[i].rt = p[i].bt;
    p[i].done = 0;
}

int time = 0, completed = 0;

// Gantt chart
char ganttPID[MAX][10];
int ganttTime[MAX];
int gIndex = 0;

int lastSysIndex = 0;

while (completed < n) {

    int executed = 0;

    // ===== SYSTEM QUEUE (RR) =====
    for (int i = 0; i < n; i++) {
        int idx = (lastSysIndex + i) % n;

        if (p[idx].type == 1 && p[idx].rt > 0 && p[idx].at <= time) {

            ganttTime[gIndex] = time;
```

```
        strcpy(ganttPID[gIndex], p[idx].pid);
        gIndex++;

        if (p[idx].rt > TQ) {
            time += TQ;
            p[idx].rt -= TQ;
        } else {
            time += p[idx].rt;
            p[idx].rt = 0;

            p[idx].ct = time;
            p[idx].tat = p[idx].ct - p[idx].at;
            p[idx].wt = p[idx].tat - p[idx].bt;

            p[idx].done = 1;
            completed++;
        }

        lastSysIndex = (idx + 1) % n;
        executed = 1;
        break;
    }
}

if (executed) continue;

// ===== USER QUEUE (FCFS) =====
int earliest = -1;

for (int i = 0; i < n; i++) {
    if (p[i].type == 2 && p[i].rt > 0 && p[i].at <= time) {
```

```
        if (earliest == -1 || p[i].at < p[earliest].at) {
            earliest = i;
        }
    }
}

if (earliest != -1) {

    ganttTime[gIndex] = time;
    strcpy(ganttPID[gIndex], p[earliest].pid);
    gIndex++;

    // Run until:
    // 1) finishes OR
    // 2) system process arrives

    int nextSysArrival = 1e9;

    for (int i = 0; i < n; i++) {
        if (p[i].type == 1 && p[i].rt > 0 && p[i].at > time) {
            if (p[i].at < nextSysArrival)
                nextSysArrival = p[i].at;
        }
    }

    if (time + p[earliest].rt <= nextSysArrival) {
        time += p[earliest].rt;
        p[earliest].rt = 0;

        p[earliest].ct = time;
        p[earliest].tat = p[earliest].ct - p[earliest].at;
```

```
            p[earliest].wt = p[earliest].tat - p[earliest].bt;

            p[earliest].done = 1;
            completed++;
        } else {
            int execTime = nextSysArrival - time;
            p[earliest].rt -= execTime;
            time = nextSysArrival;
        }

        continue;
    }

    // Idle
    time++;
}

ganttTime[gIndex] = time;

// ===== TABLE =====
printf("\n=========== MULTI-LEVEL QUEUE SCHEDULING ===========\n");
printf("+------+------+----+----+----+-----+----+\n");
printf("| PID  |TYPE | AT | BT | CT | TAT | WT |\n");
printf("+------+------+----+----+----+-----+----+\n");

float total_tat = 0, total_wt = 0;

for (int i = 0; i < n; i++) {
    printf("| %-4s | %-4s | %-2d | %-2d | %-2d | %-3d | %-2d |\n",
            p[i].pid,
            p[i].type == 1 ? "SYS" : "USR",
```

```
        p[i].at,
        p[i].bt,
        p[i].ct,
        p[i].tat,
        p[i].wt);

    total_tat += p[i].tat;
    total_wt += p[i].wt;
}

printf("+------+------+----+----+----+-----+----+\n");

printf("\nAverage Turnaround Time = %.2f\n", total_tat / n);
printf("Average Waiting Time    = %.2f\n", total_wt / n);

// ===== GANTT CHART =====
printf("\n============ GANTT CHART ============\n\n");

for (int i = 0; i < gIndex; i++) {
    printf("| %s ", ganttPID[i]);
}
printf("|\n");

for (int i = 0; i < gIndex; i++) {
    printf("+----");
}
printf("+\n");

for (int i = 0; i <= gIndex; i++) {
    printf("%-4d", ganttTime[i]);
}
```

```
    printf("\n");


    return 0;
}
```

Output:

```
Enter number of processes: 4

Process 1
Enter PID: P1
Enter Arrival Time: 0
Enter Burst Time: 4
Enter Type (1=System, 2=User): 1

Process 2
Enter PID: P2
Enter Arrival Time: 0
Enter Burst Time: 3
Enter Type (1=System, 2=User): 1

Process 3
Enter PID: P3
Enter Arrival Time: 0
Enter Burst Time: 8
Enter Type (1=System, 2=User): 2

Process 4
Enter PID: P4
Enter Arrival Time: 10
Enter Burst Time: 5
Enter Type (1=System, 2=User): 1
```

```
=========== MULTI-LEVEL QUEUE SCHEDULING ===========
+------+------+----+----+----+-----+----+
| PID  |TYPE | AT | BT | CT | TAT | WT |
+------+------+----+----+----+-----+----+
| P1   | SYS  | 0  | 4  | 6  | 6   | 2  |
| P2   | SYS  | 0  | 3  | 7  | 7   | 4  |
| P3   | USR  | 0  | 8  | 20 | 20  | 12 |
| P4   | SYS  | 10 | 5  | 15 | 5   | 0  |
+------+------+----+----+----+-----+----+

Average Turnaround Time = 9.50
Average Waiting Time    = 4.50

============ GANTT CHART ============

| P1 | P2 | P1 | P2 | P3 | P4 | P4 | P4 | P3 |
+----+----+----+----+----+----+----+----+----+
0   2   4   6   7   10  12  14  15  20
PS C:\Users\Parv Jain\OneDrive\Desktop\OS Lab>
```